DevPulse

Capstone Reference Implementation — MCP + Agent + LangGraph + A2A, on the Real GitHub API

Introduction

DevPulse is a capstone reference implementation that ties together every protocol and framework covered separately in earlier labs — MCP, a single ReAct agent, LangGraph multi-agent orchestration, and an A2A Agent Card — into one working system backed by the real, live GitHub REST API. Rather than mock data, every tool call in DevPulse genuinely hits api.github.com, so students see real responses, real rate limits, and real error handling instead of canned examples.

The project is structured in five phases that build on each other: a real MCP server exposing GitHub tools (Phase 1), shared LLM/MCP infrastructure that works with either a local or cloud model (Phase 2), a single human-in-the-loop ReAct agent (Phase 3), a LangGraph router with three specialist agents sharing that same MCP server (Phase 4), and finally an A2A Agent Card that publishes DevPulse for discovery by completely external agents (Phase 5). The codebase also documents two real engineering bugs encountered during development — both genuinely instructive, not just 'it worked first try.'

Components

Component	Description
MCP Server (mcp_server.py)	A real FastMCP server exposing 5 GitHub tools, 1 resource, and 1 prompt, backed by live HTTP calls.
MCP Client Test (mcp_client_test.py)	A bare-bones client proving initialize -> list_tools -> call_tool with no LLM involved.
Model Client (model_client.py)	Backend-agnostic LLM connector: local Ollama/Qwen by default, or cloud Groq.
MCP Bridge (mcp_bridge.py)	Shared helper converting MCP tool schemas into OpenAI-style function-calling format.
Agent Core (agent_core.py)	A single ReAct agent: Model + Tools + Memory + Human-in-the-Loop approval, talking to the real MCP server.
LangGraph Agent (langgraph_agent.py)	Multi-agent version: a router node + 3 specialist nodes, all sharing the same real MCP server.
A2A Card (a2a_card.py)	Publishes DevPulse as a discoverable Agent Card for external, framework-agnostic agents.
GITHUB_TOKEN	Optional environment variable that raises GitHub's rate limit from 60 to 5,000 requests/hour.

Syntax

```
# -- Backend-agnostic LLM client (model_client.py) ----------------------
from model_client import get_client_and_model, check_connection
client, model, backend = get_client_and_model()   # Ollama by default, or Groq
check_connection(client, model, backend)           # friendly pre-flight check

# -- MCP server (mcp_server.py) -----------------------------------------
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("DevPulse GitHub Intelligence Server")

@mcp.tool()
def get_repo_details(owner: str, repo: str) -> str:
    """Docstring becomes the tool description an LLM reads."""
    ...
mcp.run()

# -- MCP bridge: turn MCP tools into OpenAI function-calling format --------
from mcp_bridge import mcp_session, discover_openai_tools, call_mcp_tool
async with mcp_session() as session:
    tools = await discover_openai_tools(session)
    response = client.chat.completions.create(
        model=model, messages=messages, tools=tools, tool_choice="auto")
    result_text = await call_mcp_tool(session, tool_name, arguments)

# -- LangGraph router + specialists (langgraph_agent.py) -------------------
from langgraph.graph import StateGraph
builder = StateGraph(DevPulseState)
builder.add_node("classify_intent", classify_intent)
builder.add_conditional_edges("classify_intent", route,
    {"repo_info": "repo_info", "issue_triage": "issue_triage", "release_notes":
"release_notes"})

# -- A2A Agent Card (a2a_card.py) ----------------------------------------
from pydantic import BaseModel
DEVPULSE_CARD = AgentCard(name="DevPulse", skills=[Skill(id="repo_info", tags=[...])])
registry.discover("triage")   # -> ['DevPulse']
```

Learning Outcomes

- Build and run a real MCP server backed by a live external API, with proper stdout/stderr discipline.
- Write a backend-agnostic LLM client that switches between local Ollama and cloud Groq via environment variables.
- Bridge MCP tool schemas into OpenAI-style function-calling format so any LLM client can use them.
- Implement a single ReAct agent with conversational memory and human-in-the-loop tool approval.
- Build a LangGraph multi-agent system where a router dispatches to scoped specialist nodes sharing one MCP server.

- Publish an agent as a discoverable A2A Agent Card, decoupled from its internal implementation.
- Debug real integration issues: environment inheritance in subprocesses, and stdout/stdio protocol corruption.

Software Requirements

Requirement	Detail
Python Version	3.10 or higher
Core Packages	mcp>=1.0.0, httpx>=0.27.0, langgraph>=0.2.0, openai>=1.40.0
Other Libraries	python-dotenv>=1.0.0, rich>=13.0.0, pydantic>=2.0.0 (for the A2A stretch goal)
LLM Backend	Ollama (default, serving qwen2.5:3b) or Groq (cloud, set GROQ_API_KEY)
GITHUB_TOKEN	Optional but recommended — free at github.com/settings/tokens, raises rate limit 60 -> 5,000/hour
Environment File	Copy .env.example to .env and fill in any keys you plan to use
Install Command	pip install -r requirements.txt

Capstone Phases

Phase 1: Real MCP Server & Protocol Test — GitHub Intelligence Server

Source Code — mcp_server.py

```
import os
import httpx
from mcp.server.fastmcp import FastMCP

# -- SETUP ---------------------------------------------------------------
mcp = FastMCP("DevPulse GitHub Intelligence Server")

GITHUB_API = "https://api.github.com"
GITHUB_TOKEN = os.environ.get("GITHUB_TOKEN")  # optional, raises rate limit 60 -> 5000/hr

# GitHub requires a User-Agent header on every request, or it rejects the call.
_HEADERS = {"User-Agent": "DevPulse-MCP-Server"}
if GITHUB_TOKEN:
    _HEADERS["Authorization"] = f"Bearer {GITHUB_TOKEN}"


def _get(path: str, params: dict | None = None) -> tuple[int, dict]:
    """Single shared real HTTP GET against the GitHub REST API."""
    try:
        r = httpx.get(f"{GITHUB_API}{path}", headers=_HEADERS, params=params,
                      timeout=10, follow_redirects=True)
```

```
        return r.status_code, (r.json() if r.content else {})
    except httpx.RequestError as e:
        return -1, {"message": f"Network error: {e}"}


def _friendly_error(status: int, data: dict, context: str) -> str:
    if status == 403 and "rate limit" in str(data.get("message", "")).lower():
        return (f"ERROR: GitHub API rate limit exceeded. {context} "
                f"Unauthenticated requests are capped at 60/hour -- set the GITHUB_TOKEN "
                f"environment variable to raise this to 5,000/hour.")
    if status == 404:
        return f"ERROR: {context} not found on GitHub (404)."
    if status == -1:
        return f"ERROR: could not reach GitHub. {data.get('message')}"
    return f"ERROR: GitHub API returned status {status} for {context}: {data.get('message',
'unknown error')}"


# -- TOOLS (real, live actions) ----------------------------------------

@mcp.tool()
def search_repositories(query: str) -> str:
    """Search public GitHub repositories by keyword, topic, or language.
    Args: query e.g. 'machine learning language:python'
    """
    status, data = _get("/search/repositories", params={"q": query, "sort": "stars",
"per_page": 5})
    if status != 200:
        return _friendly_error(status, data, f"search for '{query}'")
    items = data.get("items", [])
    if not items:
        return f"No repositories found for query: {query}"
    lines = [f"Top results for '{query}':"]
    for repo in items:
        lines.append(f"- {repo['full_name']} *{repo['stargazers_count']} --
{repo['description']}")
    return "\n".join(lines)


@mcp.tool()
def get_repo_details(owner: str, repo: str) -> str:
    """Get live details for a GitHub repository: stars, forks, license, description.
    Args: owner e.g. 'facebook', repo e.g. 'react'
    """
    status, data = _get(f"/repos/{owner}/{repo}")
    if status != 200:
        return _friendly_error(status, data, f"{owner}/{repo}")
    license_name = (data.get("license") or {}).get("name", "No license")
    return (f"{data['full_name']}: {data.get('description', 'No description')}\n"
            f"{data['stargazers_count']:,} stars | {data['forks_count']:,} forks | "
            f"Language: {data.get('language', 'N/A')} | License: {license_name}\n"
            f"Open issues: {data['open_issues_count']} | Default branch:
{data['default_branch']}")
```

```
@mcp.tool()
def list_open_issues(owner: str, repo: str, limit: int = 5) -> str:
    """List currently open issues for a GitHub repository.
    Args: owner, repo, limit (max issues to return, default 5)
    """
    status, data = _get(f"/repos/{owner}/{repo}/issues", params={"state": "open", "per_page":
limit})
    if status != 200:
        return _friendly_error(status, data, f"{owner}/{repo} issues")
    if not data:
        return f"{owner}/{repo} currently has no open issues."
    lines = [f"Open issues for {owner}/{repo} (showing {len(data)}):"]
    for issue in data:
        labels = ", ".join(l["name"] for l in issue.get("labels", [])) or "no labels"
        lines.append(f"- #{issue['number']}: {issue['title']} [{labels}]")
    return "\n".join(lines)


@mcp.tool()
def list_contributors(owner: str, repo: str, limit: int = 5) -> str:
    """List the top contributors to a GitHub repository by commit count.
    Args: owner, repo, limit (max contributors to return, default 5)
    """
    status, data = _get(f"/repos/{owner}/{repo}/contributors", params={"per_page": limit})
    if status != 200:
        return _friendly_error(status, data, f"{owner}/{repo} contributors")
    if not data:
        return f"No contributor data available for {owner}/{repo}."
    lines = [f"Top contributors to {owner}/{repo}:"]
    for c in data:
        lines.append(f"- {c['login']}: {c['contributions']} commits")
    return "\n".join(lines)


@mcp.tool()
def get_latest_release(owner: str, repo: str) -> str:
    """Get the latest published release for a GitHub repository.
    Args: owner, repo
    """
    status, data = _get(f"/repos/{owner}/{repo}/releases/latest")
    if status != 200:
        return _friendly_error(status, data, f"{owner}/{repo} latest release")
    return (f"Latest release of {owner}/{repo}: {data['tag_name']} ({data.get('name', '')})\
n"
            f"Published: {data['published_at']}\n"
            f"Notes: {(data.get('body') or 'No release notes.')[:300]}")


# -- RESOURCE (read-only data, addressed by URI) ----------------------------

@mcp.resource("github://repo/{owner}/{repo}/summary")
def repo_summary_resource(owner: str, repo: str) -> str:
    """A pre-formatted, read-only snapshot of a repository's key stats."""
    status, data = _get(f"/repos/{owner}/{repo}")
    if status != 200:
```

```
        return _friendly_error(status, data, f"{owner}/{repo}")
    return (f"REPO SUMMARY: {data['full_name']}\n"
            f"Stars: {data['stargazers_count']} | Forks: {data['forks_count']} | "
            f"Open Issues: {data['open_issues_count']}\n"
            f"Created: {data['created_at']} | Last push: {data['pushed_at']}")


# -- PROMPT (reusable instruction template) --------------------------------

@mcp.prompt()
def issue_triage_prompt() -> str:
    """A structured workflow for triaging a GitHub issue."""
    return """
    Review this GitHub issue and:
    1. Classify it as exactly one of: Bug / Feature Request / Question / Documentation
    2. Assign a priority: Critical / High / Medium / Low, based on impact described
    3. Suggest which existing label(s) it should carry
    4. Write a one-sentence summary suitable for a triage dashboard
    Be concise -- output should fit in 4 short lines.
    """


# -- RUN THE SERVER --------------------------------------------------------
# NOTE: every line below logs to stderr, never stdout -- stdout is reserved
# exclusively for MCP's JSON-RPC protocol messages on the stdio transport.

if __name__ == "__main__":
    import sys
    print("Starting DevPulse MCP Server...", file=sys.stderr)
    print("5 tools: search_repositories, get_repo_details, list_open_issues, "
          "list_contributors, get_latest_release", file=sys.stderr)
    print("1 resource: github://repo/{owner}/{repo}/summary", file=sys.stderr)
    print("1 prompt: issue_triage_prompt", file=sys.stderr)
    if not GITHUB_TOKEN:
        print("NOTE: no GITHUB_TOKEN set -- limited to 60 unauthenticated requests/hour.",
file=sys.stderr)
    print("Waiting for MCP client connections (Ctrl+C to stop)...\n", file=sys.stderr)
    mcp.run()
```

Source Code — mcp_client_test.py

```
import asyncio
import os
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

SERVER_PATH = os.path.join(os.path.dirname(__file__), "mcp_server.py")


async def main():
    # IMPORTANT: by default, StdioServerParameters does NOT inherit the parent
    # process's environment variables. Without passing env=os.environ explicitly,
    # the server subprocess won't see GITHUB_TOKEN (or, in some sandboxes, even
    # basic things like SSL_CERT_FILE) -- a common gotcha worth teaching.
```

```
    import sys
    server_params = StdioServerParameters(
        command=sys.executable, args=[SERVER_PATH], env=os.environ.copy()
    )

    print("Launching mcp_server.py as a subprocess and connecting...")
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:

            # 1. HANDSHAKE
            await session.initialize()
            print("[1] initialize() complete -- MCP handshake done.\n")

            # 2. DISCOVERY
            tools_result = await session.list_tools()
            print(f"[2] list_tools() discovered {len(tools_result.tools)} tools:")
            for t in tools_result.tools:
                print(f"    - {t.name}: {t.description}")
            print()

            resources_result = await session.list_resources()
            print(f"    Resources: {[r.uri for r in resources_result.resources] if
resources_result.resources else '(template-based, not listed directly)'}")

            prompts_result = await session.list_prompts()
            print(f"    Prompts: {[p.name for p in prompts_result.prompts]}\n")

            # 3. INVOCATION -- real call_tool against the live GitHub API
            print("[3] call_tool('search_repositories', ...) -- a REAL call to
api.github.com:")
            result = await session.call_tool(
                "search_repositories", arguments={"query": "agentic ai framework"}
            )
            for item in result.content:
                if hasattr(item, "text"):
                    print(item.text)


if __name__ == "__main__":
    asyncio.run(main())
```

Sample Input / Output

```
python3 -c "import mcp_server as s; print(s.get_repo_details('facebook','react'))"
python3 mcp_client_test.py

--- Output ---
facebook/react: A declarative, efficient, and flexible
JavaScript library for building user interfaces.
228,000 stars | 46,800 forks | Language: JavaScript | License: MIT License
Open issues: 1,021 | Default branch: main

[1] initialize() complete -- MCP handshake done.
[2] list_tools() discovered 5 tools:
```

```
    - search_repositories: Search public GitHub repositories...
    - get_repo_details: Get live details for a GitHub repository...
[3] call_tool('search_repositories', ...) -- a REAL call to api.github.com:
Top results for 'agentic ai framework':
- microsoft/autogen *45000 -- A programming framework for agentic AI
```

Key Concepts Covered

Concept	Description
Plain Function Baseline	Calling get_repo_details() directly proves the tool works, but an LLM cannot reach it — no protocol, no discovery.
Real Live HTTP Tool	Every MCP tool here makes a genuine httpx.get() call to api.github.com, not mock data.
initialize -> list_tools -> call_tool	mcp_client_test.py proves the actual MCP protocol sequence works, independent of any LLM.
stdout vs stderr Discipline	All server logging goes to stderr; stdout is reserved exclusively for MCP's JSON-RPC messages.
env=os.environ.copy()	Required so the server subprocess inherits GITHUB_TOKEN — stdio transport does not pass environment variables by default.

Phase 2: Shared Infrastructure — Model Client & MCP Bridge

Source Code — model_client.py

```
import os
import sys


def get_client_and_model():
    """Returns (client, model_name, backend). client exposes .chat.completions.create(...)
    with the same signature regardless of backend."""
    backend = os.environ.get("LLM_BACKEND", "ollama" if not os.environ.get("GROQ_API_KEY")
else "groq").lower()

    if backend == "groq":
        from groq import Groq
        client = Groq(api_key=os.environ["GROQ_API_KEY"])
        model = os.environ.get("GROQ_MODEL", "llama-3.1-8b-instant")
        return client, model, "groq"

    # -- Ollama, via its OpenAI-compatible endpoint --------------------------
    # Same base_url pattern used by the AutoGen/LangGraph labs' config_list:
    # http://localhost:11434/v1 -- the api_key is a required-but-unchecked
    # placeholder, exactly like in those labs.
    from openai import OpenAI
    base_url = os.environ.get("OLLAMA_BASE_URL", "http://localhost:11434/v1")
    client = OpenAI(base_url=base_url, api_key="ollama")
    model = os.environ.get("OLLAMA_MODEL", "qwen2.5:3b")
    return client, model, "ollama"


def check_connection(client, model, backend):
    """Friendly pre-flight check, same idea as the MCP capstone's call_qwen
    test -- fail with a clear, backend-specific message instead of a raw
    connection traceback."""
    try:
        client.chat.completions.create(
            model=model, messages=[{"role": "user", "content": "Say OK in one word"}],
max_tokens=5
        )
        print("after api call")
        return True
    except Exception as e:
        if backend == "ollama":
            print(f"\nCould not reach Ollama at the configured base_url.", file=sys.stderr)
            print(f"  1) Is it running?  ->  ollama serve", file=sys.stderr)
            print(f"  2) Is the model pulled?  ->  ollama pull {model}", file=sys.stderr)
        else:
            print(f"\nCould not reach Groq.", file=sys.stderr)
            print(f"  Check that GROQ_API_KEY is set correctly.", file=sys.stderr)
        print(f"  Underlying error: {e}\n", file=sys.stderr)
        return False
```

Source Code — mcp_bridge.py

```
import os
import json
from contextlib import asynccontextmanager
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

SERVER_PATH = os.path.join(os.path.dirname(__file__), "mcp_server.py")


@asynccontextmanager
async def mcp_session():
    """Launch mcp_server.py as a subprocess and yield a ready ClientSession.

    NOTE: env=os.environ.copy() is required -- the MCP stdio transport does
    NOT inherit the parent process's environment by default, so without this
    the server subprocess would never see GITHUB_TOKEN.
    """
    server_params = StdioServerParameters(
        command="python", args=[SERVER_PATH], env=os.environ.copy()
    )
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            yield session


async def discover_openai_tools(session: ClientSession) -> list[dict]:
    """Real MCP list_tools(), converted into OpenAI/Groq function-calling schema.

    MCP's own tool.inputSchema is already JSON Schema, so this is a thin
    reshape rather than a hand-written schema -- proving the tool definitions
    really do come from the live server, not from a hard-coded list here.
    """
    tools_result = await session.list_tools()
    openai_tools = []
    for t in tools_result.tools:
        openai_tools.append({
            "type": "function",
            "function": {
                "name": t.name,
                "description": t.description or "",
                "parameters": t.inputSchema or {"type": "object", "properties": {}},
            },
        })
    return openai_tools


async def call_mcp_tool(session: ClientSession, name: str, arguments: dict) -> str:
    """Real call_tool() against the live MCP server, flattened to plain text."""
    result = await session.call_tool(name, arguments=arguments)
    texts = [item.text for item in result.content if hasattr(item, "text")]
    return "\n".join(texts) if texts else "(empty result)"
```

Sample Input / Output

```
client, model, backend = get_client_and_model()
check_connection(client, model, backend)

--- Output ---
# With Ollama running and qwen2.5:3b pulled:
after api call
True

# With Ollama not running:
Could not reach Ollama at the configured base_url.
  1) Is it running?  ->  ollama serve
  2) Is the model pulled?  ->  ollama pull qwen2.5:3b
```

Key Concepts Covered

Concept	Description
Backend-Agnostic Client	get_client_and_model() returns the same chat.completions.create() interface whether the backend is local Ollama or cloud Groq.
Environment-Driven Configuration	LLM_BACKEND, OLLAMA_MODEL, and GROQ_API_KEY switch behavior with zero code changes.
Pre-Flight Connection Check	check_connection() turns a raw network traceback into a clear, actionable, backend-specific error message.
MCP-to-OpenAI Schema Bridge	discover_openai_tools() reshapes MCP's native tool.inputSchema into the OpenAI function-calling format — no hand-written schemas.
Framework-Agnostic Bridge	mcp_session()/call_mcp_tool() work identically whether called from a plain script, LangGraph, CrewAI, or AutoGen.

Phase 3: Single ReAct Agent with Human-in-the-Loop

Source Code — agent_core.py

```
"""
============================================================
DevPulse -- Agent Core
Model + Tools + Memory + Human-in-the-Loop (HITL)

Flow:
User -> LLM (Reason) -> Select Tool -> Human Approval ->
MCP Tool -> GitHub API -> LLM Response
============================================================
"""

import json
import asyncio

from model_client import get_client_and_model, check_connection
from mcp_bridge import mcp_session, discover_openai_tools, call_mcp_tool


SYSTEM_PROMPT = (
    "You are DevPulse, an assistant that answers questions about GitHub "
    "repositories using real, live tools. Always use a tool to look up facts "
    "instead of guessing. Remember repo names mentioned earlier in the "
    "conversation so the user doesn't have to repeat them."
)


class DevPulseAgent:

    def __init__(self):
        self.client, self.model, self.backend = get_client_and_model()
        self.history = []

    async def ask(self, user_input: str, max_loops: int = 4) -> str:

        self.history.append({"role": "user", "content": user_input})

        async with mcp_session() as session:

            # Discover MCP tools dynamically
            tools = await discover_openai_tools(session)

            for _ in range(max_loops):

                messages = [{"role": "system", "content": SYSTEM_PROMPT}] + self.history

                response = self.client.chat.completions.create(
                    model=self.model, messages=messages, tools=tools, tool_choice="auto"
                )

                msg = response.choices[0].message
                finish_reason = response.choices[0].finish_reason
```

```
                # No tool required -> Final Answer
                if finish_reason != "tool_calls" or not msg.tool_calls:
                    self.history.append({"role": "assistant", "content": msg.content})
                    return msg.content

                # Store assistant tool request
                self.history.append({
                    "role": "assistant",
                    "content": msg.content,
                    "tool_calls": [tc.model_dump() for tc in msg.tool_calls]
                })

                # Execute each tool requested by the LLM
                for tool_call in msg.tool_calls:

                    args = json.loads(tool_call.function.arguments)

                    print("\n" + "=" * 60)
                    print(" HUMAN-IN-THE-LOOP APPROVAL ")
                    print("=" * 60)
                    print(f"Tool Selected : {tool_call.function.name}")
                    print(f"Arguments     : {args}")
                    print("=" * 60)

                    approval = input("Approve tool execution? (y/n): ").strip().lower()

                    if approval != "y":
                        print("\nTool execution rejected by human.\n")
                        self.history.append({
                            "role": "tool", "tool_call_id": tool_call.id,
                            "content": "Tool execution denied by human reviewer."
                        })
                        continue

                    print("\nHuman approved.")
                    print("Executing MCP Tool...\n")

                    result_text = await call_mcp_tool(session, tool_call.function.name, args)

                    print("=" * 60)
                    print(" TOOL RESULT ")
                    print("=" * 60)
                    print(result_text)
                    print("=" * 60)

                    # Feed tool result back to LLM
                    self.history.append({
                        "role": "tool", "tool_call_id": tool_call.id, "content": result_text,
                    })

        return "I wasn't able to resolve this within the allotted reasoning steps."


async def main():
```

```
    agent = DevPulseAgent()

    if not check_connection(agent.client, agent.model, agent.backend):
        return

    print("\n" + "=" * 70)
    print(" DevPulse Agent with Human-in-the-Loop ")
    print("=" * 70)
    print(f"Model : {agent.model}")
    print("Type 'quit' to exit.")
    print("=" * 70)

    while True:

        user_input = input("\nYou : ").strip()

        if user_input.lower() == "quit":
            break
        if not user_input:
            continue

        answer = await agent.ask(user_input)

        print("\n" + "=" * 70)
        print(" DevPulse Response ")
        print("=" * 70)
        print(answer)
        print("=" * 70)


if __name__ == "__main__":
    asyncio.run(main())
```

Sample Input / Output

```
You : Tell me about the repo facebook/react
You : now show me its contributors

--- Output ---
============================================================
 HUMAN-IN-THE-LOOP APPROVAL
============================================================
Tool Selected : get_repo_details
Arguments     : {'owner': 'facebook', 'repo': 'react'}
============================================================
Approve tool execution? (y/n): y

Human approved.
Executing MCP Tool...
============================================================
 TOOL RESULT
============================================================
facebook/react: A declarative, efficient, and flexible...
```

```
============================================================
 DevPulse Response
============================================================
facebook/react has 228,000 stars and 46,800 forks...
```

Key Concepts Covered

Concept	Description
ReAct Loop with a Cap	ask() loops up to max_loops times: reason, select a tool, act, observe, repeat — bounded so it can't run forever.
Human-in-the-Loop Approval	Every tool call pauses for an explicit (y/n) human approval before it touches the real GitHub API.
Conversational Memory	self.history persists across ask() calls, letting 'its contributors' resolve to the repo named in the prior turn.
Dynamic Tool Discovery	tools = await discover_openai_tools(session) means the agent's tool list always reflects the live MCP server, not a hard-coded list.
Rejection Path	If the human says 'n', a denial message is fed back to the LLM as the tool result instead of executing anything.

Phase 4: Multi-Agent LangGraph Version — Router + 3 Specialists

Source Code — langgraph_agent.py

```
"""
============================================================
DevPulse -- LangGraph Multi-Agent Version
Router node classifies intent -> conditional edges dispatch to
one of 3 specialist nodes -> each calls the REAL MCP server.
============================================================
"""

import os
import json
import asyncio
from typing import TypedDict, Optional
from langgraph.graph import StateGraph
from model_client import get_client_and_model, check_connection
from mcp_bridge import mcp_session, discover_openai_tools, call_mcp_tool

client, MODEL, BACKEND = get_client_and_model()


class DevPulseState(TypedDict):
    query: str
    history: list           # short-term memory across turns
    category: str            # repo_info | issue_triage | release_notes
    tool_calls_made: list
    answer: str


# -- ROUTER NODE: classify intent ------------------------------------------

def classify_intent(state: DevPulseState) -> DevPulseState:
    prompt = (
        "Classify this GitHub-related query into exactly one category:\n"
        "repo_info, issue_triage, release_notes\n\n"
        "repo_info = general repo stats, search, contributors\n"
        "issue_triage = anything about open issues\n"
        "release_notes = anything about latest release/version\n\n"
        f"Query: {state['query']}\n\nReply with ONLY one of the three words."
    )
    result = client.chat.completions.create(model=MODEL, messages=[{"role": "user",
"content": prompt}])
    category = result.choices[0].message.content.strip().lower()
    valid = {"repo_info", "issue_triage", "release_notes"}
    state["category"] = category if category in valid else "repo_info"
    print(f"  [router] classified as: {state['category']}")
    return state


def route(state: DevPulseState) -> str:
    return state["category"]


# -- SHARED SPECIALIST LOGIC: each node runs its own scoped ReAct mini-loop ---
```

```
async def _run_specialist(state: DevPulseState, persona: str, allowed_tools: set[str]) ->
DevPulseState:
    """Every specialist shares one real MCP session but is restricted to its
    own subset of tools -- this is what makes it a 'specialist' rather than
    just three copies of the same general agent."""
    async with mcp_session() as session:
        all_tools = await discover_openai_tools(session)
        scoped_tools = [t for t in all_tools if t["function"]["name"] in allowed_tools]

        messages = [{"role": "system", "content": persona}] + state["history"] + [
            {"role": "user", "content": state["query"]}
        ]
        response = client.chat.completions.create(
            model=MODEL, messages=messages, tools=scoped_tools, tool_choice="auto"
        )
        msg = response.choices[0].message

        if response.choices[0].finish_reason == "tool_calls" and msg.tool_calls:
            for tc in msg.tool_calls:
                args = json.loads(tc.function.arguments)
                print(f"  [{persona.split()[1]}] {tc.function.name}({args})")
                tool_result = await call_mcp_tool(session, tc.function.name, args)
                state["tool_calls_made"].append({"tool": tc.function.name, "result":
tool_result})
            # Second pass: synthesize the final answer from real tool result(s)
            follow_up = messages + [
                {"role": "assistant", "content": msg.content, "tool_calls": [tc.model_dump()
for tc in msg.tool_calls]}
            ] + [
                {"role": "tool", "tool_call_id": tc.id, "content": call["result"]}
                for tc, call in zip(msg.tool_calls, state["tool_calls_made"][-
len(msg.tool_calls):])
            ]
            final = client.chat.completions.create(model=MODEL, messages=follow_up)
            state["answer"] = final.choices[0].message.content
        else:
            state["answer"] = msg.content

        state["history"].append({"role": "user", "content": state["query"]})
        state["history"].append({"role": "assistant", "content": state["answer"]})
    return state


def repo_analyst_node(state: DevPulseState) -> DevPulseState:
    persona = ("You are the Repo Analyst. Use search_repositories or get_repo_details "
               "or list_contributors to answer with real, current GitHub data.")
    return asyncio.run(_run_specialist(
        state, persona, {"search_repositories", "get_repo_details", "list_contributors"}
    ))


def issue_triage_node(state: DevPulseState) -> DevPulseState:
    persona = ("You are the Issue Triage Agent. Use list_open_issues to answer with "
               "real, currently-open issues. Briefly note priority where relevant.")
```

```
    return asyncio.run(_run_specialist(state, persona, {"list_open_issues"}))


def release_notes_node(state: DevPulseState) -> DevPulseState:
    persona = ("You are the Release Notes Agent. Use get_latest_release to answer "
               "with the real, current latest version and notes.")
    return asyncio.run(_run_specialist(state, persona, {"get_latest_release"}))


# -- BUILD THE GRAPH -------------------------------------------------------

builder = StateGraph(DevPulseState)
builder.add_node("classify_intent", classify_intent)
builder.add_node("repo_info", repo_analyst_node)
builder.add_node("issue_triage", issue_triage_node)
builder.add_node("release_notes", release_notes_node)

builder.add_conditional_edges(
    "classify_intent", route,
    {"repo_info": "repo_info", "issue_triage": "issue_triage", "release_notes":
"release_notes"},
)
builder.set_entry_point("classify_intent")
builder.set_finish_point("repo_info")
builder.set_finish_point("issue_triage")
builder.set_finish_point("release_notes")

graph = builder.compile()


def main():
    if not check_connection(client, MODEL, BACKEND):
        return
    print(f"DevPulse (LangGraph multi-agent) ready (model: {MODEL}). Type 'quit' to exit.\n")
    history = []
    while True:
        query = input("You: ").strip()
        if query.lower() == "quit":
            break
        if not query:
            continue
        result = graph.invoke({
            "query": query, "history": history, "category": "",
            "tool_calls_made": [], "answer": "",
        })
        history = result["history"]
        print(f"DevPulse [{result['category']}]: {result['answer']}\n")


if __name__ == "__main__":
    main()
```

Sample Input / Output

```
You: What are the open issues on facebook/react?

--- Output ---
  [router] classified as: issue_triage
  [Issue] list_open_issues({'owner': 'facebook', 'repo': 'react'})
DevPulse [issue_triage]: facebook/react currently has several
open issues, including #28910 (bug, needs triage) and
#28905 (documentation)...
```

Key Concepts Covered

Concept	Description
Conditional-Edge Routing	classify_intent() + route() send each query to exactly one of three specialist nodes via add_conditional_edges().
Scoped Specialist Agents	_run_specialist() restricts each node to its own allowed_tools subset of the same shared MCP session — this is what makes each node a 'specialist'.
Two-Pass Tool Synthesis	Each specialist calls the LLM once to select a tool, then a second time to turn the real tool result into a natural-language answer.
Short-Term Memory in State	history is carried inside DevPulseState and returned to the REPL loop, persisting context across turns even with three different specialist nodes.
One MCP Server, One Source of Truth	All three specialists and the router ultimately go through the same live mcp_server.py — no direct GitHub calls anywhere else.

Phase 5: A2A Agent Card — Publishing DevPulse for External Discovery

Source Code — a2a_card.py

```
from pydantic import BaseModel
from typing import List


class Skill(BaseModel):
    id: str
    description: str
    tags: List[str]


class AgentCard(BaseModel):
    name: str
    description: str
    version: str
    endpoint: str
    skills: List[Skill]


# -- DevPulse's own self-description ---------------------------------------
# Each skill maps 1:1 to one of the LangGraph specialist nodes in
# langgraph_agent.py. An external agent only ever sees this card -- never
# the MCP server, the GitHub API, or the LangGraph code underneath it.

DEVPULSE_CARD = AgentCard(
    name="DevPulse",
    description="Agentic GitHub intelligence assistant backed by a real MCP server.",
    version="1.0.0",
    endpoint="http://localhost:9001/devpulse",  # where it would be reachable, if deployed
    skills=[
        Skill(
            id="repo_info",
            description="Look up live stats, search, and contributors for a GitHub
repository",
            tags=["github", "repository", "stars", "contributors", "search"],
        ),
        Skill(
            id="issue_triage",
            description="List and triage open issues for a GitHub repository",
            tags=["github", "issues", "triage", "bugs"],
        ),
        Skill(
            id="release_notes",
            description="Get the latest release/version info for a GitHub repository",
            tags=["github", "release", "version", "changelog"],
        ),
    ],
)


# -- Minimal registry, same shape as the A2A lab manual ----------------------

class AgentRegistry:
```

```
    def __init__(self):
        self.agents: list[AgentCard] = []

    def register(self, card: AgentCard):
        self.agents.append(card)

    def discover(self, tag: str) -> list[str]:
        return [a.name for a in self.agents for s in a.skills if tag in s.tags]


def demo():
    registry = AgentRegistry()
    registry.register(DEVPULSE_CARD)

    print("Registered agents:")
    for a in registry.agents:
        print(f"  - {a.name}: {[s.id for s in a.skills]}")

    print("\nAn external agent searching for who can triage GitHub issues:")
    matches = registry.discover("triage")
    print(f"  discover('triage') -> {matches}")

    print("\nAn external agent searching for release/version info:")
    matches = registry.discover("release")
    print(f"  discover('release') -> {matches}")

    print("\nFull card as published (this is all an external agent ever sees):")
    print(DEVPULSE_CARD.model_dump_json(indent=2))


if __name__ == "__main__":
    demo()
```

Sample Input / Output

```
python3 a2a_card.py

--- Output ---
Registered agents:
  - DevPulse: ['repo_info', 'issue_triage', 'release_notes']

An external agent searching for who can triage GitHub issues:
  discover('triage') -> ['DevPulse']

An external agent searching for release/version info:
  discover('release') -> ['DevPulse']

Full card as published (this is all an external agent ever sees):
{
  "name": "DevPulse",
  "description": "Agentic GitHub intelligence assistant backed by a real MCP server.",
  ...
}
```

Key Concepts Covered

Concept	Description
Agent Card as the Only Visible Surface	An external agent sees only DEVPULSE_CARD — never the MCP server, GitHub API, or LangGraph code underneath.
1:1 Skill-to-Node Mapping	Each Skill (repo_info, issue_triage, release_notes) corresponds directly to one specialist node in langgraph_agent.py.
MCP vs A2A Boundary	MCP solves agent-to-tool access (GitHub data); A2A solves agent-to-agent discovery — DevPulse only needed A2A once it had to be callable by an unrelated external system.
Discovery by Skill Tag	registry.discover('triage') demonstrates exactly how an outside agent would find DevPulse without knowing anything about its internals.

Cheat Sheet

Syntax / Call	Purpose
get_client_and_model()	Return an (client, model, backend) tuple for either Ollama or Groq.
check_connection(client, model, backend)	Pre-flight test that fails with a clear, actionable error message.
mcp_session()	Async context manager launching the MCP server subprocess and yielding a ready ClientSession.
discover_openai_tools(session)	Convert live MCP tools into the OpenAI function-calling schema.
call_mcp_tool(session, name, arguments)	Invoke a real MCP tool and flatten its result to plain text.
client.chat.completions.create(..., tools=..., tool_choice='auto')	Ask the LLM to reason and optionally select a tool to call.
StdioServerParameters(command, args, env=os.environ.copy())	Launch an MCP server subprocess while preserving environment variables.
builder.add_conditional_edges(node, router_fn, mapping)	Route a LangGraph node's output to one of several specialist nodes.
AgentCard(name, description, skills=[...])	Publish an agent's capabilities for external A2A discovery.
registry.discover(tag)	Find which registered agent(s) advertise a given skill tag.

Industry Use Cases

Pattern	Real-World Application
Real MCP server over a live API	Internal developer tools, DevOps copilots wired to real GitHub/Jira/CI systems.
Backend-agnostic LLM client	Cost/latency failover between a free local model and a paid cloud model in production.
Human-in-the-loop tool approval	Any agent performing sensitive or irreversible actions (deployments, financial transactions, deletions).
LangGraph router + scoped specialists	Engineering productivity assistants, support copilots that triage and route across domains.
A2A Agent Card publication	Enterprise agent marketplaces, cross-team agent discovery without exposing internal architecture.

Troubleshooting

Issue	Solution
GITHUB_TOKEN silently missing inside the MCP server	Pass env=os.environ.copy() to StdioServerParameters — the stdio transport does not inherit the parent environment by default.
'Failed to parse JSONRPC message from server'	A stray print() inside the server is writing to stdout. Send all server logging to stderr (file=sys.stderr) instead.
ERROR: GitHub API rate limit exceeded	Expected unauthenticated GitHub behavior (60/hr). Set GITHUB_TOKEN to raise the limit to 5,000/hr, or try /search/repositories which has its own separate quota.
Could not reach Ollama at the configured base_url	Run 'ollama serve' and 'ollama pull qwen2.5:3b'; check_connection() reports exactly which of these is missing.
Could not reach Groq	Confirm GROQ_API_KEY is set and exported correctly in your environment or .env file.
langgraph_agent.py specialist ignores conversation history	Confirm history is being passed in and returned correctly through DevPulseState across graph.invoke() calls.

Conclusion

DevPulse demonstrates how MCP, single-agent ReAct loops, LangGraph multi-agent orchestration, and A2A discovery fit together in one real, working system rather than as separate, disconnected exercises. By building on a live external API instead of mock data, the capstone surfaces genuine engineering concerns — environment inheritance, transport protocol discipline, and rate limiting — that only show up once a system leaves the classroom and touches the real world. The clean separation between the MCP layer (agent-to-tool) and the A2A layer (agent-to-agent) is the key architectural lesson: each protocol solves a distinct problem, and a well-designed system only reaches for the one it actually needs.